

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 5
«Процедуры, функции, триггеры в PostgreSQL»
по дисциплине «Проектирование и реализация баз данных»**

Обучающийся Митрофанова Полина Олеговна
Факультет прикладной информатики
Группа K3240
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии 2023
Преподаватель Говорова Марина Михайловна

Санкт-Петербург
2025/2026

Цель работы: овладеть практическими создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

Практическое задание:

1. Создать 3 процедуры для индивидуальной БД согласно варианту (часть 4 ЛР 2). Допустимо использование IN/OUT параметров. Допустимо создать авторские процедуры. (3 балла)

Создать хранимые процедуры:

- о текущей сумме вклада и сумме начисленного за месяц процента для заданного клиента;
- добавить данные о новом вкладе клиента;
- найти клиентов банка, не имеющих задолженности по кредитам.

2. Создать триггеры для индивидуальной БД согласно варианту:

Вариант 2.1. 3 триггера - 3 балла (min). Допустимо использовать триггеры логирования из практического занятия по функциям и триггерам.

Вариант 2.2. 7 оригинальных триггеров - 7 баллов (max).

Индивидуальное задание - Вариант 3. БД «Библиотека»

Название: БД «Библиотека»

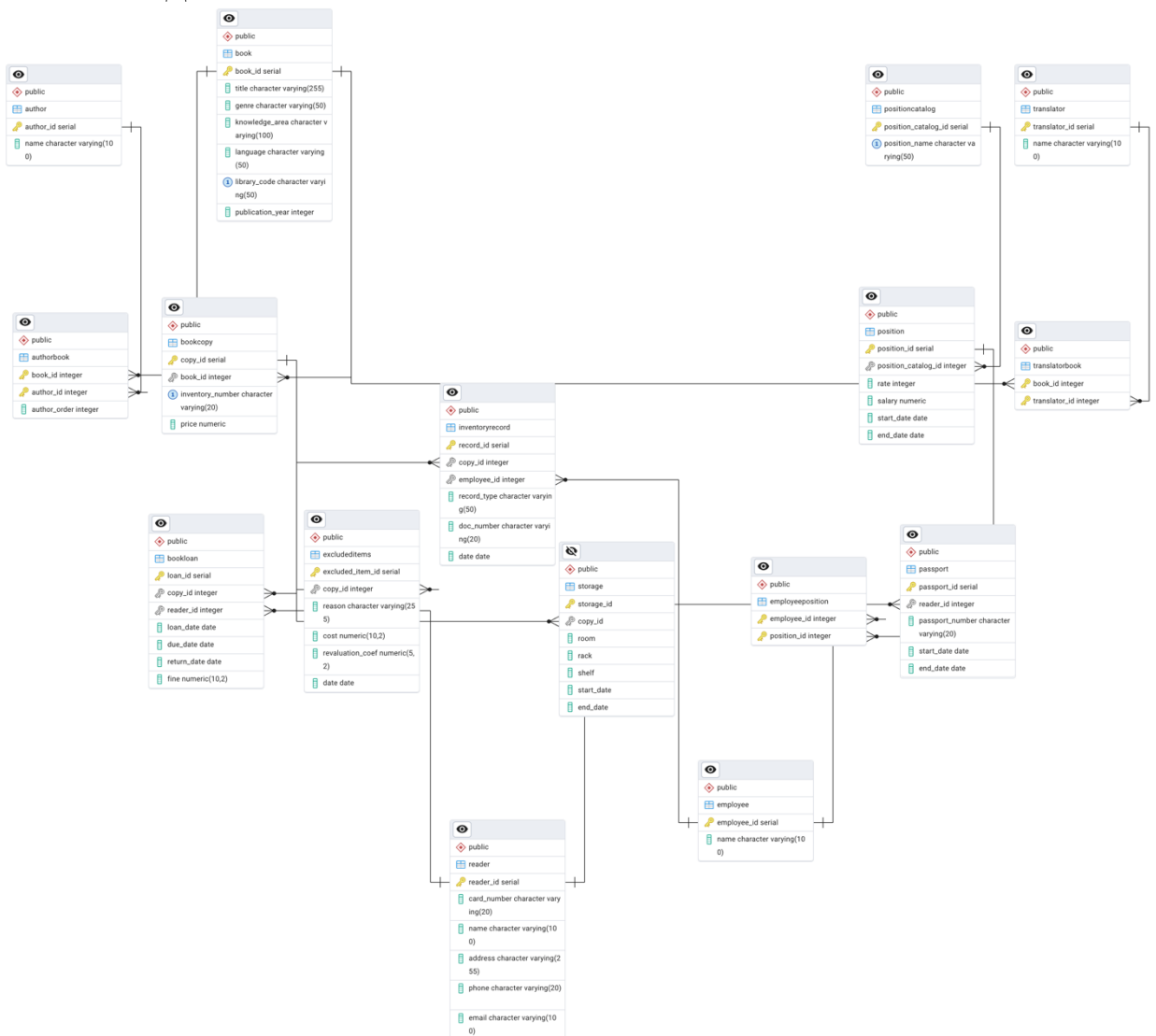


Рисунок 1 - Схема логической модели базы данных.

Выполнение:
Процедура 1

```

lab3=# CREATE OR REPLACE FUNCTION get_book_copy_count(p_book_id INT)
lab3=# RETURNS INT AS $$
lab3$$ DECLARE
lab3$#     copy_count INT;
lab3$# BEGIN
lab3$#     SELECT COUNT(*)
lab3$#     INTO copy_count
lab3$#     FROM BookCopy bc
lab3$#     LEFT JOIN ExcludedItems ei ON bc.copy_id = ei.copy_id
lab3$#     WHERE bc.book_id = p_book_id
lab3$#           AND ei.copy_id IS NULL;
lab3$#
lab3$#     RETURN copy_count;
lab3$# END;
lab3$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
lab3=# SELECT get_book_copy_count(1);
get_book_copy_count
-----
2
(1 row)

```

Процедура 2

```

polinamitrofanova@MacBookAir LR3 % docker exec -it biblio_db psql -U polina_mitrofanova -d lab3
psql (14.22)
Type "help" for help.

lab3=# CREATE OR REPLACE PROCEDURE add_new_book(
lab3(#     p_title VARCHAR,
lab3(#     p_genre VARCHAR,
lab3(#     p_knowledge_area VARCHAR,
lab3(#     p_language VARCHAR,
lab3(#     p_library_code VARCHAR,
lab3(#     p_publication_year INT
lab3(# )
lab3=# LANGUAGE plpgsql
lab3=# AS $$
lab3$# BEGIN
lab3$#     IF EXISTS (SELECT 1 FROM Book WHERE library_code = p_library_code) THEN
lab3$#         RAISE EXCEPTION 'Книга с кодом % уже существует.', p_library_code;
lab3$#     END IF;
lab3$#
lab3$#     INSERT INTO Book (title, genre, knowledge_area, language, library_code, publication_year)
lab3$#     VALUES (p_title, p_genre, p_knowledge_area, p_language, p_library_code, p_publication_year);
lab3$# END;
lab3$# $$;
CREATE PROCEDURE

lab3=# CALL add_new_book ('New book', 'Science', 'Physics', 'Russian', 'LC999', 2026);
CALL
lab3=# SELECT * FROM Book WHERE library_code = 'LC999';
 book_id | title | genre | knowledge_area | language | library_code | publication_year
-----+-----+-----+-----+-----+-----+-----
      8 | New book | Science | Physics | Russian | LC999 |          2026
(1 row)

lab3=#

```

Процедура 3

```

lab3=# CREATE OR REPLACE PROCEDURE add_new_reader(
lab3(#      p_card_number VARCHAR,
lab3(#      p_name VARCHAR,
lab3(#      p_address VARCHAR,
lab3(#      p_phone VARCHAR,
lab3(#      p_email VARCHAR
lab3(# )
lab3=# LANGUAGE plpgsql
lab3=# AS $$
lab3$# BEGIN
lab3$#      IF EXISTS (SELECT 1 FROM Reader WHERE card_number = p_card_number) THEN
lab3$#          RAISE EXCEPTION 'Читатель с номером карты % уже существует.', p_card_number;
lab3$#      END IF;
lab3$#
lab3$#      INSERT INTO Reader (card_number, name, address, phone, email)
lab3$#      VALUES (p_card_number, p_name, p_address, p_phone, p_email);
lab3$# END;
lab3$# $$;
lab3=# CREATE PROCEDURE
lab3=# CALL add_new_reader ('R1234', 'John Smith', 'NY, Penny Lane st., 8', '+79673499853', 'pizza_eater123@gmail.com');
CALL
lab3=# SELECT * FROM Reader WHERE card_number = 'R1234';
 reader_id | card_number |      name      |      address      |      phone      |      email
-----+-----+-----+-----+-----+-----
      8 | R1234      | John Smith | NY, Penny Lane st., 8 | +79673499853 | pizza_eater123@gmail.com
(1 row)

lab3=#

```

Триггер 1 – при добавлении книги — логировать в отдельную таблицу BookLog.

```

lab3=# CREATE TABLE IF NOT EXISTS BookLog (
lab3(#      log_id SERIAL PRIMARY KEY,
lab3(#      book_id INT,
lab3(#      title TEXT,
lab3(#      operation_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
lab3(# );
lab3=# CREATE TABLE
lab3=#
lab3=# CREATE OR REPLACE FUNCTION log_book_insert()
lab3=# RETURNS TRIGGER AS $$
lab3$# BEGIN
lab3$#      INSERT INTO BookLog(book_id, title)
lab3$#      VALUES (NEW.book_id, NEW.title);
lab3$#      RETURN NEW;
lab3$# END;
lab3$# $$ LANGUAGE plpgsql;
lab3=# CREATE FUNCTION
lab3=# CALL add_new_book ('Trigger', 'Thriller', 'Literature', 'Russian', 'LC_TRIG1', 2003);
CALL
lab3=# SELECT * FROM BookLog WHERE title = 'Trigger';
 log_id | book_id | title |      operation_time
-----+-----+-----+-----
      2 |      9 | Trigger | 2026-05-18 09:49:15.869847
(1 row)

```

Триггер 2 – Запрет удаления книг, если есть выданные экземпляры

```

lab3=#
lab3=# CREATE TRIGGER trg_log_book_insert
lab3=# AFTER INSERT ON Book
lab3=# FOR EACH ROW EXECUTE FUNCTION log_book_insert();
lab3=# CREATE TRIGGER
lab3=# CREATE OR REPLACE FUNCTION prevent_book_delete_if_on_loan()
lab3=# RETURNS TRIGGER AS $$
lab3$# BEGIN
lab3$#      IF EXISTS (
lab3$#          SELECT 1
lab3$#          FROM BookCopy bc
lab3$#          JOIN BookLoan bl ON bc.copy_id = bl.copy_id
lab3$#          WHERE bc.book_id = OLD.book_id AND bl.return_date IS NULL
lab3$#      ) THEN
lab3$#          RAISE EXCEPTION 'Нельзя удалить книгу: экземпляры находятся на руках у читателей.';
lab3$#      END IF;
lab3$#      RETURN OLD;
lab3$# END;
lab3$# $$ LANGUAGE plpgsql;
lab3=# CREATE FUNCTION
lab3=# DELETE FROM Book WHERE book_id = 1;
ERROR:  Нельзя удалить книгу: экземпляры находятся на руках у читателей.
CONTEXT:  PL/pgSQL function prevent_book_delete_if_on_loan() line 9 at RAISE
lab3=#

```

Триггер 3 – при обновлении return_date в BookLoan — автоматически заполнить fine

```
lab3=# CREATE TRIGGER trg_prevent_book_delete
lab3=# BEFORE DELETE ON Book
lab3=# FOR EACH ROW EXECUTE FUNCTION prevent_book_delete_if_on_loan();
CREATE TRIGGER
lab3=# CREATE OR REPLACE FUNCTION calculate_fine_on_return()
lab3=# RETURNS TRIGGER AS $$
lab3$# BEGIN
lab3$#     IF NEW.return_date IS NOT NULL AND NEW.return_date > NEW.due_date THEN
lab3$#         NEW.fine := ROUND(EXTRACT(DAY FROM (NEW.return_date - NEW.due_date)) * 5, 2);
lab3$#     ELSE
lab3$#         NEW.fine := 0;
lab3$#     END IF;
lab3$#     RETURN NEW;
lab3$# END;
lab3$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
```

Триггер 4 – логирование нового читателя

```
lab3=# CREATE TRIGGER trg_calculate_fine
lab3=# BEFORE UPDATE OF return_date ON BookLoan
lab3=# FOR EACH ROW EXECUTE FUNCTION calculate_fine_on_return();
CREATE TRIGGER
lab3=# CREATE TABLE IF NOT EXISTS ReaderLog (
lab3(#     log_id SERIAL PRIMARY KEY,
lab3(#     reader_id INT,
lab3(#     name TEXT,
lab3(#     registered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
lab3(# );
CREATE TABLE
lab3=#
lab3=# CREATE OR REPLACE FUNCTION log_new_reader()
lab3=# RETURNS TRIGGER AS $$
lab3$# BEGIN
lab3$#     INSERT INTO ReaderLog(reader_id, name)
lab3$#     VALUES (NEW.reader_id, NEW.name);
lab3$#     RETURN NEW;
lab3$# END;
lab3$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
lab3=# SELECT * FROM ReaderLog;
lab3=#
```

log_id	reader_id	name	registered_at
1	8	John Smith	2026-05-18 09:45:33.523135

(1 row)

Триггер 5 - автогенерация inventory_number при вставке копии

Триггер 6 - автоматическое завершение хранения при возврате книги

```
lab3=# CREATE TRIGGER trg_log_new_reader
lab3=# AFTER INSERT ON Reader
lab3=# FOR EACH ROW EXECUTE FUNCTION log_new_reader();
CREATE TRIGGER
lab3=# CREATE OR REPLACE FUNCTION auto_inventory_number()
lab3=# RETURNS TRIGGER AS $$
lab3$# BEGIN
lab3$#     IF NEW.inventory_number IS NULL OR NEW.inventory_number = '' THEN
lab3$#         NEW.inventory_number := 'INV' || NEW.copy_id;
lab3$#     END IF;
lab3$#     RETURN NEW;
lab3$# END;
lab3$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
```

```

lab3=# CREATE TRIGGER trg_storage_return
lab3=# AFTER UPDATE OF return_date ON BookLoan
lab3=# FOR EACH ROW WHEN (NEW.return_date IS NOT NULL)
lab3=# EXECUTE FUNCTION update_storage_on_return();
CREATE TRIGGER
lab3=# CREATE TABLE IF NOT EXISTS EmployeeDeletedLog (
lab3(#     log_id SERIAL PRIMARY KEY,
lab3(#     employee_id INT,
lab3(#     name TEXT,
lab3(#     deleted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
lab3(# );
CREATE TABLE
lab3=# CREATE TRIGGER trg_auto_inventory
lab3=# BEFORE INSERT ON BookCopy
lab3=# FOR EACH ROW EXECUTE FUNCTION auto_inventory_number();
CREATE TRIGGER
lab3=# CREATE OR REPLACE FUNCTION update_storage_on_return()
lab3=# RETURNS TRIGGER AS $$
lab3$# BEGIN
lab3$#     IF NEW.return_date IS NOT NULL THEN
lab3$#         UPDATE Storage
lab3$#         SET end_date = CURRENT_DATE
lab3$#         WHERE copy_id = NEW.copy_id AND end_date IS NULL;
lab3$#     END IF;
lab3$#     RETURN NEW;
lab3$# END;
lab3$# $$ LANGUAGE plpgsql;
CREATE FUNCTION

```

Триггер 7 - лог удаления сотрудника

```

lab3=# CREATE OR REPLACE FUNCTION log_employee_deletion()
lab3=# RETURNS TRIGGER AS $$
lab3$# BEGIN
lab3$#     INSERT INTO EmployeeDeletedLog(employee_id, name)
lab3$#     VALUES (OLD.employee_id, OLD.name);
lab3$#     RETURN OLD;
lab3$# END;
lab3$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
lab3=#
lab3=# CREATE TRIGGER trg_log_employee_delete
lab3=# AFTER DELETE ON Employee
lab3=# FOR EACH ROW EXECUTE FUNCTION log_employee_deletion();
CREATE TRIGGER

```

Вывод:

В данной лабораторной работе я познакомилась с процедурами и функциями в базах данных, научилась их писать и использовать. Также я узнала о триггерах, принципах их создания и работы. Сделала свои процедуры и триггеры под реализованную бд и протестировала их. В результате получила навыки создания надёжной базы данных.